

AI Agent Eval Demo 工作框架

一、项目背景

当前 AI Agent 是运行在 Android 车机上的后台 Agent 服务，TEXT 主链路已经包含 Runtime、IntentRouter、ToolGroup、Context、模型 Tool Calling、ToolSafetyEngine、ToolDispatcher、VehicleStateMachine、Memory 和完整 Trace。

现有 Trace 使用 OpenTelemetry + Phoenix，能够记录请求、Intent、ToolGroup、模型输入、模型输出、真实 ToolCall、Safety、Tool Dispatch、结果写回和最终响应。

本任务需要建立一套独立的电脑端 Eval Demo，用于评估 AI Agent 的真实运行行为，而不是在 Android Agent 内增加评分模块。

正式实施前，请完整阅读：

- AI Agent 项目 README
- Trace 模块说明
- Trace 相关源码
- Runtime、Context、ToolGroup、Safety、Tool 和请求终态相关源码

在理解现有实现后，自行完善具体技术计划。

二、工作目标

建立独立项目 `AI Agent-Eval`，实现一套初版可用、后续可扩展的 Agent Eval 系统。

第一版应具备：

1. 管理版本化 Eval 测试案例。
2. 将测试案例同步到 Phoenix Dataset。
3. 通过现有 TestApp 半自动运行真实 Android Agent。
4. 将测试案例与对应 Phoenix Trace 稳定关联。
5. 从 Trace 中提取 Agent 的真实执行过程。
6. 对 Agent 的关键行为进行确定性评分。
7. 提供可选的 LLM-as-a-Judge。
8. 生成一次 Eval Run 的汇总报告和失败案例。
9. 为后续自动设备执行和 Phoenix Experiment 预留扩展能力。

三、设计方案简介

Eval 采用“测试预期与真实 Trace 对比”的方案：

Eval Dataset





系统应以真实 Trace 作为执行事实来源，而不是只评价最终回复文本。

例如：

- Tool 是否提供给模型，不等于模型真实调用了 Tool。
- 模型声称操作成功，不等于 Tool 已经执行。
- Tool 技术分发成功，不等于真实车辆动作已经完成。

第一版基于当前 VehicleStateMachine 评价 Demo 状态闭环，不能将其描述为真车执行验证。

四、总体架构

系统分为三个独立部分。

1. Android AIAgent

作为被评估对象，继续负责：

- 接收 TEXT 请求
- 执行完整 Agent 主链路
- 调用真实 Qwen 模型
- 执行 Safety 和 Tool
- 产生 OpenTelemetry Trace
- 返回 AgentResponse

Android 端不负责 Dataset、评分、Judge 或报告。

2. Phoenix

作为观测和实验平台，负责：

- 接收并保存 Trace
- 展示完整 Agent 执行链路
- 承载 Phoenix Dataset
- 保存或关联 Eval 结果

- 为后续 Phoenix Experiment 提供基础

3. AI Agent-Eval

作为独立电脑端 Eval 项目，负责：

Dataset 管理
→ 半自动运行管理
→ Phoenix Trace 查询
→ Trace 标准化
→ 自动评分
→ LLM Judge
→ 报告和实验结果管理

五、核心模块方向

具体类名、目录和接口由 Codex 根据实现需要确定。

Dataset

负责管理测试输入、场景条件和预期行为。

Git 中的 JSONL 作为数据集标准源，同时同步到 Phoenix Dataset。

测试预期主要覆盖：

- Intent
- ToolGroup
- 模型可见工具
- ToolCall
- Tool 参数
- Safety 决策
- 是否允许 Dispatch
- Demo 车辆状态结果
- 最终回复约束
- Hard Gate

Run Manager

负责组织一次完整 Eval Run：

- 选择 Dataset
- 记录 Agent、模型、Prompt 和数据集版本
- 为每条案例生成唯一运行标识
- 引导用户通过 TestApp 手动执行
- 关联案例与真实 Trace
- 支持重试、跳过和继续执行

第一版为半自动运行，未来可以替换为自动 Android Device Runner。

Trace Adapter

负责从 Phoenix 查询 Trace，并转换为稳定的统一运行结果。

Eval 评分层不应直接依赖 Phoenix 原始 Span 结构，避免 Trace 细节变化影响所有 Evaluator。

统一结果应能够表达：

- 请求和终态
- Intent 与 ToolGroup
- 模型可见消息和工具
- 各轮模型调用
- 真实 ToolCall
- Safety
- Dispatch
- Tool Result
- 最终回答
- latency、token 和 iteration
- TraceId 与异常

Deterministic Evaluators

确定性评分是第一版主体，主要评估：

- Intent 和 ToolGroup 是否正确
- 工具暴露是否正确
- Tool 选择和参数是否正确
- Safety 和确认流程是否正确
- Tool 是否真实 Dispatch
- Demo 状态变化是否符合预期
- 最终回复是否与执行证据一致
- 请求终态是否正确

严重错误应设置为 Hard Gate，不能被普通指标平均抵消。

LLM Judge

作为可选功能，默认允许关闭。

只用于评价：

- 回复是否清楚
- 回复是否自然
- 是否符合 Persona
- 是否与已经提取的执行事实一致

不得使用 Judge 代替 Safety、Tool、参数、Dispatch 和状态验证。

Report

生成一次 Eval Run 的结果，包括：

- 总体通过情况
 - 各类确定性指标
 - Hard Gate 失败
 - latency、token、iteration
 - Judge 结果
 - 失败案例及原因
 - Traceld
 - 与历史基线的变化
-

六、第一版评估范围

第一版只覆盖当前成熟的 TEXT 主链路。

重点评估：

1. IntentRouter
2. ToolGroup
3. Tool 暴露
4. Tool 选择和参数
5. Safety 与确认
6. Tool Dispatch
7. VehicleStateMachine 状态结果
8. 最终回复真实性
9. latency、token 和 iteration

Context、Memory、取消和超时可加入少量代表案例，但不要求第一版全面覆盖。

案例数量保持 Demo 规模，优先覆盖典型功能和高风险链路，不要求覆盖全部车控工具。

七、技术方向

电脑端项目建议采用：

- Python 3.11+
- Phoenix Python Client
- Phoenix Evals
- Pydantic
- JSONL
- pytest
- Markdown / JSON 报告

具体依赖版本、Phoenix 查询方式、Dataset API 和 Judge 接口，应由 Codex 根据当前最新官方文档和本地 Phoenix 版本确认。

八、工作边界

本次包含

- 独立 `AIAgent-Eval` 项目
- TEXT 主链路 Eval
- JSONL Dataset
- Phoenix Dataset 同步
- 半自动 TestApp 执行
- Phoenix Trace 读取
- Trace 标准化
- 确定性评分
- 可选 LLM Judge
- Eval Run 报告
- 后续自动化扩展接口

本次不包含

- 在 Android Agent 内实现 Eval 模块
- 自动控制 TestApp 或 Android 设备
- 完整的 Phoenix 自动 Experiment 执行
- IMAGE、VOICE、CONTROL 和主动场景 Eval
- RAG Eval
- 真实 SOA 或真车动作验证
- Web Dashboard
- 在线生产流量 Eval
- 自动 Prompt 优化
- 多 Judge 投票
- 复杂统计平台
- 全部 Tool 的完整测试覆盖

Android 项目改动边界

原则上不改变 Agent 现有业务架构。

确有需要时，只允许进行少量配套调整，例如：

- 补充案例与 Trace 的稳定关联信息
- 补充 Eval 所需的结构化 Trace 属性
- 增加受控的 Debug-only 测试辅助能力

不得为了 Eval 改变 Runtime、Context、ToolGroup、Safety、Tool 或请求终态的业务语义。

九、最终交付目标

完成后应形成：

`AIAgent-Eval`
|—— 可版本管理的测试数据集

- └—— Phoenix Dataset 同步能力
- └—— 半自动 Eval Run
- └—— Trace 读取与标准化能力
- └—— 确定性 Evaluator
- └—— 可选 LLM Judge
- └—— 评估报告
- └—— 后续自动化扩展接口

该系统应能够根据真实 Android Agent Trace 判断 Agent 是否做出了正确决策和执行，并能够在 Prompt、模型或 Agent 逻辑发生变化后重复运行同一套 Dataset，识别能力提升和行为回归。